Evaluating Parallel FIR Filter and Edge Detection Workloads

Submitted To:

Professor David Kaeli
EECE 5640 – High Performance Computing – Final Project

Submitted By:

Justin Bahr

Due: Thursday April 15, 2025

1. Abstract

This paper explores how high performance computing optimizations can reduce runtimes for audio and image processing engineering workloads written in C++. Specifically, this paper examines how the SIMD architecture of AVX512 and multithreading with OpenMP affect how fast a finite impulse response filter output can be computed using a CPU. Additionally, this paper profiles the performance of a Sobel filter edge detection algorithm run with CUDA accelerated code for an NVIDIA GPU and whether tiling or data types affects this. Through testing various implementations on different hardware platforms, this experiment revealed that OpenMP and AVX support can significantly the time needed to filter and output a WAV file and that GPU acceleration provides a scalable platform to increase throughput while increasing image size to maintain optimal performance.

2. Workloads

2.1 Audio Processing - FIR Filter: The first workload is an audio processing program that applies a finite impulse response filter to a song. Finite impulse response (FIR) filters can be used to filter out specific frequency ranges (lowpass, highpass, bandpass, bandstop), apply a sound effect such as reverb, or apply equalization to sound samples. They are preferred over infinite impulse response filters for audio applications due to their linear phase response. When designing digital filters, there is a tradeoff between attaining tighter specifications and minimizing the number of calculations. The order of a digital filter corresponds to the length of the impulse response (order M=L-1). A higher order filter can provide higher stopband attenuation and/or a narrower transition bandwidth, both of which are desirable qualities. However, adding more terms means more multiplications and additions are required to calculate the resulting signal. The filter designed for this workload is a 191 order equiripple low pass filter designed using the Parks-McClellan Algorithm in MATLAB. Frequencies under 880 Hz (A5 note on piano) will pass through unaffected and frequencies over 1360 Hz (about F6 on the piano) will be attenuated by at least 50 decibels. Visual references for this filter are shown below:

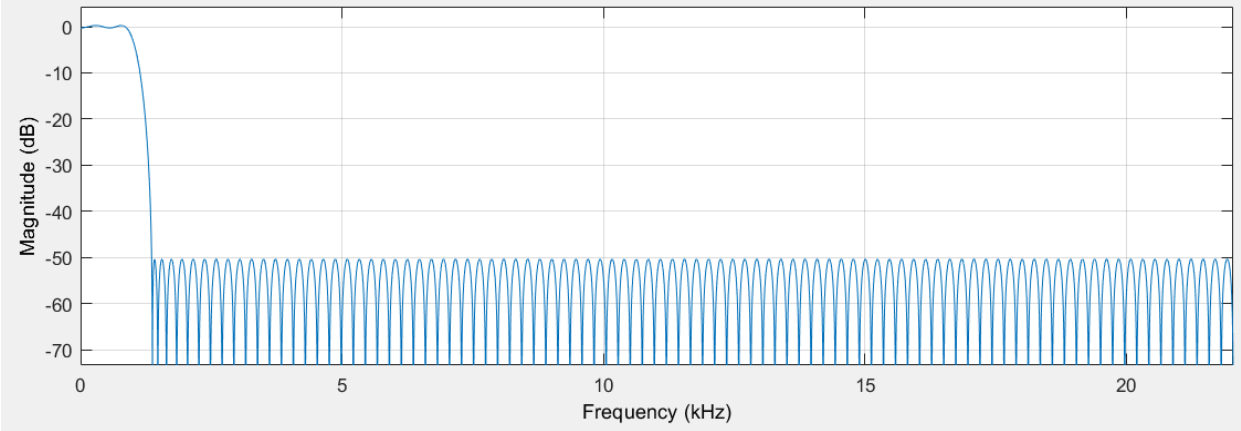

Figure 1. Log-Magnitude Response of the FIR Filter

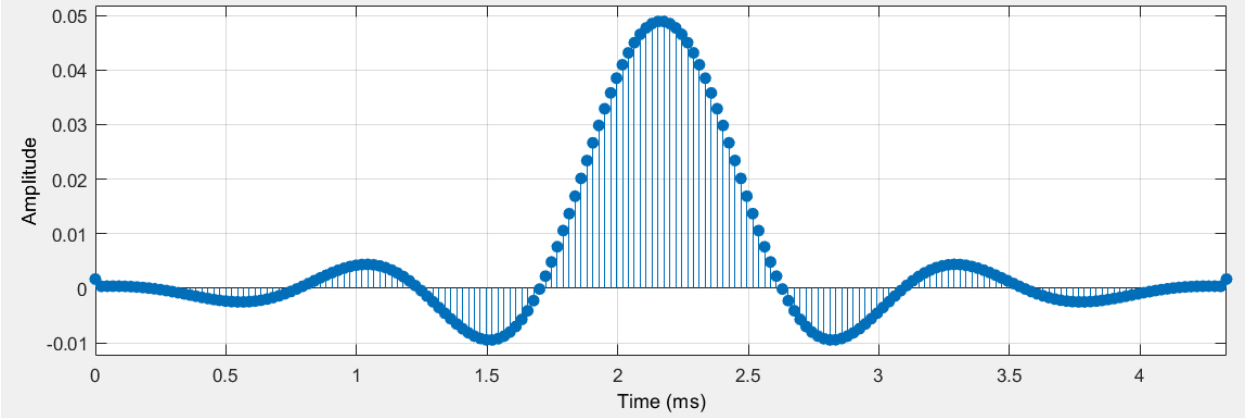

Figure 2. Impulse Response of the FIR Filter

An FIR filter is executed using linear convolution, which essentially glides a series of filter coefficients along the signal and sums the results. Coefficients stored as doubles are first converted to single precision floats to improve performance, especially for the AVX optimizations. The standard, single-threaded code body for this operation is shown below:

```
// function to perform FIR filtration
void FIR_lowpass(const int16_t inputL[], int16_t outputL[], const int16_t inputR[], int16_t outputR[],
    int signalLength,const float coefficients[], int order)
{
    // filters left channel
    for (int i = 0; i < signalLength; i++)
    {
        float temp = 0.0;
        for (int j = 0; j < min(order,i+1); j++)
        {
            temp += coefficients[j] * inputL[i-j];
        }
        outputL[i] = (int16_t)(temp);
    }

    // filters right channel
    for (int i = 0; i < signalLength; i++)
    {
        float temp = 0.0;
        for (int j = 0; j < min(order,i+1); j++)
        {
            temp += coefficients[j] * inputR[i-j];
        }
        outputR[i] = (int16_t)(temp);
    }
}
```

Figure 3. Serial FIR Convolution Code Body

Another sub-workload within this program is converting between interleaved stereo data to individual left and right channel data arrays. The code body for this sub-workload is shown below:

```
// fills the input stereo data
for (int i = 0; i < inputLength; i++)
{
    inputL[i] = data[2*i];
    inputR[i] = data[2*i + 1];
}
```

```
// fills the output data array
for (int i = 0; i < inputLength; i++)
{
    data[2*i] = outputL[i];
    data[2*i+1] = outputR[i];
}
```

Figure 4. Stereo Data Conversion Cody Body

2.2 Edge Detection - Sobel Filter: The second workload is an image processing program that performs edge detection using a Sobel filter. The Sobel filter works by convolving an image with two 3x3 kernels to estimate the gradient in both the X and Y directions. The X and Y Sobel operators are shown below:

-1	0	+1
-2	0	+2
-1	0	+1

Gx

+1	+2	+1
0	0	0
-1	-2	-1

Gy

Figure 5. Sobel Operators

Based on the gradients in both dimensions, a grayscale image highlighting the edge can be produced by assigning lighter colors to pixels with a high gradient, and darker colors pixels with a lower gradient. Below is an example of before and after a Sobel filter edge detection algorithm is applied:

Figure 6. Edge Detection Visualization

3. Optimization Techniques

3.1a FIR Filter Optimization - OpenMP: Regardless of whether a minimum order filter is being used, a long input audio sample will require many calculations, and thus take a long time to filter. FIR filters are implemented through linear convolutions, which are an easily parallelizable operation, lending well to high performance computing optimizations. Each output sample is independent of other output samples, so each output sample can be calculated based on the input array and the coefficient array, which are both known at the start of the function call. Therefore, the main computational loops can be parallelized simply using OpenMP directives as shown below:

```
#pragma omp parallel
{
    // filters left channel
    #pragma omp for nowait
    for (int i = 0; i < signalLength; i++)
    {
        float temp = 0.0;
        for (int j = 0; j < min(order,i+1); j++)
        {
            temp += coefficients[j] * inputL[i-j];
        }
        outputL[i] = (int16_t)(temp);
    }
    // filters right channel
    #pragma omp for nowait
    for (int i = 0; i < signalLength; i++)
    {
        float temp = 0.0;
        for (int j = 0; j < min(order,i+1); j++)
        {
            temp += coefficients[j] * inputR[i-j];
        }
        outputR[i] = (int16_t)(temp);
    }
}
```

Figure 7. FIR Filter Parallelized with OpenMP

Beyond the filter loops, the loop splitting the interleaved stereo input data into left and right channels and the loop interleaving left and right channeled output data can be parallelized using OpenMP as shown below:

```
// fills the input stereo data
#pragma omp parallel for
for (int i = 0; i < inputLength; i++)
{
    inputL[i] = data[2*i];
    inputR[i] = data[2*i + 1];
} // end parallel section
```

```
// fills the output data array
#pragma omp parallel for
for (int i = 0; i < inputLength; i++)
{
    data[2*i] = outputL[i];
    data[2*i+1] = outputR[i];
} // end parallel section
```

Figure 8. Parallel Input/Output Sequencing

3.1b FIR Filter Optimization - AVX512: Additionally, input samples and coefficients can be vectorized since each output term of the convolution is essentially a dot product of the reversed coefficient vector, and sample vector of terms equal to the length of the coefficient vector, but offset to the to the left so that the last term of the reversed coefficient vector (first term of the original coefficient list), is multiplied by the current sample. To avoid flipping the coefficient list, it is important to note that a low-pass FIR filter is symmetric, so its reversal is equivalent to the original list. This is true for all type I and II FIR filters. Furthermore, vector instructions require that the operands are of the same data type, so the 16-bit integer samples must first be converted to single precision floats before being loaded into vector registers to match the filter coefficients. With these two qualifiers in mind, 16 pairs of samples and coefficients can be loaded into the 512-bit vector registers and the multiplications can be done with a single AVX instruction thanks to SIMD architecture. Next, the 16 results are accumulated and added to a temporary float variable that accumulates the full result before storing it in the output array. An example of this is shown below:

```
// filter the rest of the left channel with AVX
for (int i = order - 1; i < signalLength; i++)
{
    __m512 sumL = _mm512_setzero_ps();

    int j = 0;
    // processes in chunks of 16 coefficients
    for (; j + 15 < order; j += 16)
    {
        // loads coefficients
        __m512 coeffs = _mm512_loadu_ps(&coefficients[j]);

        // Load 16 int16 values and convert to float
        __m256i inputVals = _mm256_loadu_si256((__m256i*)&inputL[i - order + j + 1]);
        __m512 inputsL = _mm512_cvtepi32_ps(_mm512_cvtepi16_epi32(inputVals));

        sumL = _mm512_fmadd_ps(coeffs, inputsL, sumL); // Multiply-accumulate
    }

    // Reduce sum to scalar
    float tempL = _mm512_reduce_add_ps(sumL);

    // Remainder loop (if order is not a multiple of 16)
    for (; j < min(order, i + 1); j++)
    {
        tempL += coefficients[j] * inputL[i - j];
    }

    // Store output, converting back to int16
    outputL[i] = static_cast<int16_t>(tempL);
}
```

Figure 9. FIR Filter using AVX512

Another complication to note is that early samples with an index less than the filter order cannot be vectorized in this way since only some of the filter coefficients are used to compute these terms. These terms are dealt with at the beginning and only make up a small fraction of the samples, so the performance loss is not significant. Code for this process is shown below:

```
// deals with early values in the left channel
for (int i = 0; i < order - 1; i++)
{
    float tempL = 0;
    for (int j = 0; j < i + 1; j++)
    {
        tempL += coefficients[j] * inputL[i - j];
    }
    outputL[i] = static_cast<int16_t>(tempL);
}
```

Figure 10. Early Term Processing

3.1c Combining OpenMP and AVX FIR Filter Optimizations: Each output sample is independent, therefore, each can be computed concurrently. Added AVX support does not prevent any of the parallelizations implemented in 3.1a. To combine these optimizations, OpenMP directive are added to the AVX optimized code as shown below:

```
// deals with early values in the left channel
#pragma omp for
for (int i = 0; i < order - 1; i++)
{
    float tempL = 0;
    for (int j = 0; j < i + 1; j++)
    {
        tempL += coefficients[j] * inputL[i - j];
    }
    outputL[i] = static_cast<int16_t>(tempL);
}

// filters remaining left channel
#pragma omp for
for (int i = 0; i < signalLength; i++)
{
    __m512 sumL = _mm512_setzero_ps();

    int j = 0;
    // processes in chunks of 16 coefficients
    for (; j + 15 < min(order, i + 1); j += 16)
    {
        // loads coefficients
        __m512 coeffs = _mm512_loadu_ps(&coefficients[j]);

        // Load 16 int16 values and convert to float
        __m256i inputVals = _mm256_loadu_si256((__m256i*)&inputL[i - order + j + 1]);
        __m512 inputsL = _mm512_cvtepi32_ps(_mm512_cvtepi16_epi32(inputVals));

        sumL = _mm512_fmadd_ps(coeffs, inputsL, sumL); // Multiply-accumulate
    }

    // Reduce sum to scalar
    float tempL = _mm512_reduce_add_ps(sumL);

    // Remainder loop (if order is not a multiple of 8)
    for (; j < min(order, i + 1); j++)
    {
        tempL += coefficients[j] * inputL[i - j];
    }

    // Store output, converting back to int16
    outputL[i] = static_cast<int16_t>(tempL);
}
```

Figure 11. FIR Filter using AVX512 Parallelized with OpenMP

3.2a Sobel Filter Edge Detection - CUDA Acceleration: The grid size chosen to launch kernels was 16x16. Two CUDA kernels are used to accelerate this code. The first converts an RGB image to grayscale using a standard formula, and the second applies the Sobel filter, which uses 2 dimensional convolution with two 3x3 matrices to calculate the X and Y gradients.

```
// CUDA kernel to convert RGB to Grayscale
__global__ void rgbToGrayscale(unsigned char *rgb, unsigned char *gray, int width, int height)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < width && y < height)
    {
        int idx = (y * width + x) * 3; // RGB pixel index
        gray[y * width + x] = (unsigned char)(0.299f * rgb[idx] + 0.587f * rgb[idx + 1] + 0.114f * rgb[idx + 2]);
    }
}
```

Figure 12. RGB to Grayscale CUDA Kernel

```
// CUDA kernel for Sobel edge detection
__global__ void sobelFilter(unsigned char *input, unsigned char *output, int width, int height)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x > 0 && x < width - 1 && y > 0 && y < height - 1)
    {
        int Gx = 0, Gy = 0;

        // applies Sobel operators to pixels
        for (int i = -1; i <= 1; i++)
        {
            for (int j = -1; j <= 1; j++)
            {
                int pixel = input[(y + i) * width + (x + j)];
                Gx += pixel * SOBEL_X[i + 1][j + 1];
                Gy += pixel * SOBEL_Y[i + 1][j + 1];
            }
        }

        // computes the msgnitude of the gradient
        int magnitude = sqrtf(Gx * Gx + Gy * Gy);
        output[y * width + x] = (magnitude > 255) ? 255 : magnitude;
    }
}
```

Figure 13. Sobel Filter CUDA Kernel

3.2b Sobel Filter Edge Detection Optimization - Tiling: Shared memory tiling was added to both the RGB to grayscale CUDA kernel as well as the Sobel filter kernel. In the first CUDA kernel, the tiles are 16x16x3, holding a 16x16 group of pixels, each with a red, blue, and green value. This ratio allows for a good balance of concurrent blocks, and shared memory per block. This tiled RGB to Grayscale kernel is shown in figure 14. In the second CUDA kernel uses a tile size of 18x18, which stores a 16x16 group of grayscale pixels and a one pixel wide halo around this square to store edge values calculated from the Sobel operator matrix. Individual results are then combined after a __syncthreads() call is made to ensure all threads have completed their processes before results are accumulated. A screenshot of the shared memory tile used is shown in figure 15.

```
// CUDA kernel to convert RGB to Grayscale in tiles
__global__ void rgbToGrayscaleTiled(unsigned char *rgb, unsigned char *gray, int width, int height)
{
    // defines the tile and shared memory size
    __shared__ unsigned char tile_rgb[16][16][3]; // RGB channels

    // stores indices
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int x = blockIdx.x * 16 + tx;
    int y = blockIdx.y * 16 + ty;

    int idx = (y * width + x) * 3;

    if (x < width && y < height)
    {
        // loads RGB data into shared memory
        tile_rgb[ty][tx][0] = rgb[idx];     // R
        tile_rgb[ty][tx][1] = rgb[idx + 1]; // G
        tile_rgb[ty][tx][2] = rgb[idx + 2]; // B
    }

    __syncthreads();

    if (x < width && y < height)
    {
        unsigned char r = tile_rgb[ty][tx][0];
        unsigned char g = tile_rgb[ty][tx][1];
        unsigned char b = tile_rgb[ty][tx][2];

        // computes the grayscale value
        gray[y * width + x] = (unsigned char)(0.299f * r + 0.587f * g + 0.114f * b);
    }
}
```

Figure 14. Tiled RGB to Grayscale CUDA Kernel

```
// defines the tile size with a 1-pixel halo
__shared__ unsigned char tile[16 + 2][16 + 2];
```

Figure 15. Shared Memory Tile for the Sobel CUDA Kernel

3.2c Sobel Filter Edge Detection Optimization - uchar3: Similar to dim3, uchar3 is a group of three 8-bit unsigned characters, which is perfect for an RGB pixel, since the red, green, and blue values are all stored as 8-bit unsigned characters. Loading RGB data to the GPU this way allows for more memory coalescing and works well with the dimensional structure of the GPU. The CUDA code added to utilize this data type is shown below:

```
size_t rgbSize = width * height * sizeof(uchar3);;
size_t graySize = width * height;

uchar3 *d_rgb; // creates a pointer for an RGB pixel
unsigned char *d_gray, *d_output;

// allocates memory on the GPU
cudaMalloc((void **)&d_rgb, rgbSize);
cudaMalloc((void **)&d_gray, graySize);
cudaMalloc((void **)&d_output, graySize);

// copies RGB data to the GPU
cudaMemcpy(d_rgb, (uchar3 *)h_rgbData, rgbSize, cudaMemcpyHostToDevice);
```

Figure 16. Loading RGB Values as uchar3 Data Types

```
if (x < width && y < height)
{
    int idx = y * width + x;
    uchar3 pixel = rgb[idx]; // loads R, G, and B values into a single pixel data type
    gray[idx] = (unsigned char)(0.299f * pixel.x + 0.587f * pixel.y + 0.114f * pixel.z);
}
```

Figure 17. Indexing the X, Y, and Z Pixel Dimensions to Compute Grayscale Value

4. Datasets

4.1 Audio Dataset: The audio samples used are from the Star Wars movies (Main Title, Princess Leia's Theme, The Imperial March, Anakin's Dream, Duel of the Fates, and Yoda's Theme). These songs make up a good dataset for low pass filtering because they have a wide range of frequencies, and they are long enough to obtain noticeable speedups in parallel. The MP3 files used are available here: https://archive.org/details/13BinarySunsetAlternate. The MP3 files were then converted into stereo (2-channel) WAV files for processing. The WAV header information can be parsed to obtain information about the sample rate, sample length, audio format, and storage size. In order to properly parse the WAV data, the LIST information must be removed from the WAV file binaries. An example is shown here:

```
LISTÞNULNULNULINFOIARTSONULNULNULJohn Williams
```

Figure 18. WAV File LIST Data

4.2 Image Dataset: PPM files are fairly easy to parse in C++, so this file type was used for inputs to the edge detection workload, and PGM files (the grayscale equivalent) were used for the output images. The set of four images ranging from 640x426 to 5184x3456 pixels used in this experiment can be found here: https://filesamples.com/formats/ppm. The image consists of a field with blue waves and a forest backdrop as seen below:

Figure 19. Edge Detection Image Sample

5. Testing Methodology

5.1 Github Repository: All source files for this project (input samples not included due to file size restrictions) can be found here: https://github.com/JustinBahr305/EECE5640_FinalProject.git

5.1a Audio Processing Executables and Hardware Platforms: Four versions of the FIR filter audio processing workload were created in C++ (source code found in the Audio directory):

Executable Name	Implementation
fir	Single-threaded
fir_omp	Parallelized using OpenMP
fir_avx	Vectorized using AVX512
fir_combo	Combination of AVX and OpenMP

Figure 20. FIR Filter Program Versions

All four executables were tested on an AVX512 supported node on the Explorer Cluster (Intel ® Xeon ® Platinum 8276 CPU @ 2.20GHz, 2 sockets, 28 cores per socket). Both fir and fir_omp were tested on a node without AVX512 support on the Explorer Cluster (Intel ® Xeon ® CPU E5-2680 v4 @2.40GHz, 2 sockets, 14 cores per socket) and a node on the COE Vector System (Intel ® Xeon ® CPU E5-2698 v4 @ 2.20GHz, 2 sockets, 20 cores per socket).

5.1b Audio Processing Workload Timing Metrics: Results were timed using the high precision clock provided by the chrono library. The total program runtime as well as the time to filter and time to read/write each audio file was recorded in nanoseconds. The total program runtime was used to determine which implementation executed the fastest and calculate the total speedup. The individual filter and read/write time stamps were used to determine whether OpenMP can significantly reduce the time needed to split and interleave stereo data while reading and writing WAV files. These timestamps were also used to determine what percent of the runtime is taken up by reading and writing WAV files.

5.1c Anticipated Results for the Audio Processing Workloads: The anticipated order from slowest to fastest runtime for executables was fir, fir_avx, fir_omp, fir_combo. This is because AVX512 allows for a maximum of a 16x theoretical speedup since 16 floats

can fit in the 512 bit vector register, but when AVX is used, the 16-bit integer operands must be converted to 32-bits floats. Also, some early samples cannot be vectorized and there are other associated performance costs. The OpenMP implementation was expected to run faster than the AVX512 implementations since all CPU nodes used have more than 16 cores available, and there are less performance costs associated with multithreading compared to AVX512 for this application. Finally the combination of AVX and OpenMP was expected to produce the fastest result by combining the benefits of both optimizations without scaling either down, utilizing many AVX512 cores to maximize compute throughput. It was also expected that the COE Vector system will achieve a larger speedup due to OpenMP since these nodes have 40 cores compared to only 28 on the Explorer Cluster CPU nodes. In terms of the read/write runtime, it is anticipated that using OpenMP to split and interleave stereo data will cause a noticeable speedup, but that reading and writing WAV files will take up no more than 10% of the runtime for each audio sample.

5.2a Edge Detection Executables and Hardware Platforms: There is one C++ source file for all workloads, but three different CUDA extension source files were used for the Sobel filter kernels (source code in the Edge_Detection directory):

Executable Name	Implementation
edge	Default Kernel
edge_tiled	Tiled Kernel
edge_uchar3	Kernel Loads Pixels as uchar3

Figure 21. Sobel Kernel Version

All three executables were tested on a GPU-based node under the courses-gpu partition on the Explorer Cluster. This node has an Intel ® Xeon ® CPU E5-2680 v4 @ 2.40GHz host and an NVIDIA Tesla P100 PCIe 12GB GPU device.

5.2b Edge Detection Workload Timing Metrics: Results were timed using the high precision clock provided by the chrono library. The total program runtime as well as the time to process each image size was recorded in nanoseconds. The total program runtime was used to determine which implementation executed the fastest, and the individual runtimes for different image sizes were used to evaluate scalability.

5.2c Anticipated Results for the Edge Detection Workloads: The anticipated order from slowest to fastest runtime for executables was edge, edge_tiled, edge_uchar3. Tiling with shared memory should improve performance since it has quicker latency than global memory. Using uchar3 data types to load RGB values should also improve performance by loading the R, G, and B values to the GPU device.

6. Results

6.1a Performance of the Audio Workload on CPU Node on the Explorer Cluster with AVX512 Support Testing onXeon Platinum 8276 CPU proved the anticipated order of executables by runtime was correct. The single-thread implementation (fir) took an average of 134.06 seconds to process all samples. The AVX implementation (fir_avx) took an average of 22.03 seconds to process all samples. The OpenMP implementation (fir_omp) took an average 3.82 seconds to process all samples. The combined implementation (fir_combo) took an average of 1.26 seconds to process all samples. The respective speedups were 6.08x for AVX512 alone, 35.14x for OpenMP alone, and 106.47x for the combined approach. Detailed results for all trials are shown below:

Explorer AVX	fir	fir_omp	fir_avx	fir_combo
Trial 1	133.8083167	4.446532115	21.99581446	1.311000144
Trial 2	133.8749969	3.608250078	22.01632936	1.209145067
Trial 3	133.7824145	3.573499133	21.99609705	1.21622243
Trial 4	133.8549607	3.8068995	22.14048873	1.30685285
Trial 5	134.9787388	3.642552194	22.02458676	1.252397866
Average	134.0598855	3.815546604	22.03466327	1.259123671

Figure 22. FIR Runtime in Seconds on an AVX512 CPU Node

6.1b Performance of the Audio Workload on a CPU Node on the Explorer Cluster Without AVX512 Support: On the Xeon E5-2680 CPU, the average runtime for fir was 147.41 seconds single-threaded and 6.38 seconds with multi-threading via OpenMP. Parallelizing with OpenMP produced a 23.09x speedup. Compared to the Xeon Platinum 8276 CPU, the Xeon E5-2680 CPU took 1.06x as long to process the samples serially, and 1.67x as long in parallel. Detailed results for all trials are shown below:

Explorer	fir	fir_omp
Trial 1	147.3708617	6.800484617
Trial 2	147.6307263	6.186035274
Trial 3	146.9333287	6.170592517
Trial 4	146.8691008	6.181477974
Trial 5	148.2537977	6.584256332
Average	147.411563	6.384569343

Figure 23. FIR Runtime in Seconds on an Explorer Node

6.1c Performance of the Audio Workload on a CPU Node on the COE Vector System (no AVX512 Support): On the Xeon E5-2698 CPU, the average runtime for fir was 117.18 seconds single-threaded and 6.50 seconds with multi-threading via OpenMP. The serial implementation ran 1.26x faster compared to the Xeon E5-2680 CPU, and the both parallel implementations were comparable in terms of runtime. Detailed results for all trials are shown below:

COE Vector	fir	fir_omp
Trial 1	119.2728741	7.75043612
Trial 2	116.7119812	4.702131813
Trial 3	116.6283366	4.672602407
Trial 4	116.6943561	9.34039656
Trial 5	116.6082375	6.040177628
Average	117.1831571	6.501148906

Figure 24. FIR Runtime in Seconds on a COE Vector Node

6.1d Performance of Reading/Writing WAV Files Compared to Filtering Contents: On the Explorer Cluster, using OpenMP to parallelize the stereo data parsing resulted in a 2.60x speedup compared to without OpenMP. Reading and writing WAV files took on average 1.8% of runtime per audio sample without OpenMP, but took on average 15.6% of runtime per audio sample with OpenMP applied. Detailed results are shown below:

Explorer	Read/Write	Time (ns)		Filter	Time (ns)
	Serial	OpenMP		Serial	OpenMP
StarWars4	582590608	239892892		35429760863	1331184424
StarWars6	425594793	187104380		24523796790	916700120
StarWars10	334097611	155630466		16790596256	714133943
StarWars12	462260633	162515745		26202232013	981717921
StarWars13	505310107	168354394		23349087261	876691391
StarWars20	357552751	111607325		19274673855	724581143
Sum	2667406503	1025105202		145570147038	5545008942
OpenMP Speedup	2.60208074			26.25246389	

Figure 25. Read/Write vs Filter Runtime on Explorer Cluster

On the COE Vector System, using OpenMP to parallelize stereo data parsing resulted in double the runtime (slower than the single-threaded implementation). Reading and writing WAV files took on average 1.18% of runtime per audio sample without OpenMP, but took on average 31.94% of runtime per audio sample with OpenMP applied. Detailed results are shown below:

COE Vector	Read/Write	Time (ns)		Filter	Time (ns)
	Serial	OpenMP		Serial	OpenMP
StarWars4	454639873	656052582		29496157786	1493322219
StarWars6	215908041	375411008		19518265023	773552578
StarWars10	146181926	452869726		13357921182	561353646
StarWars12	227393268	491515169		20850715444	1147960066
StarWars13	197547803	522187010		18589274547	1072909961
StarWars20	161399321	313029867		15353993808	940582488
Sum	1403070232	2811065362		117166327790	5989680958
OpenMP Speedup	0.4991240157			19.56136372	

Figure 26. Read/Write vs Filter Runtime on COE Vector

6.1e Output WAV File Correctness: Each implementation executed on every CPU node in this experiment produced an identical set of results. The low pass filtration is evident when listening to the output WAV files. Only low frequencies at or below 880 Hz are present, and the resulting samples sound similar to if music was listened to under water or through a thick wall. Additionally, the frequency responses were plotted in MATLAB to confirm:

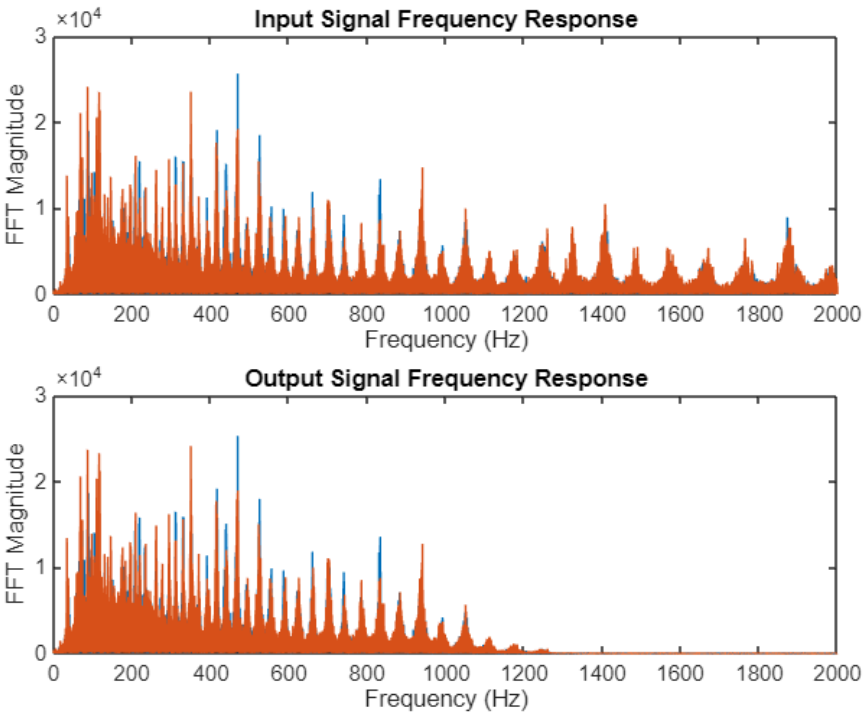

Figure 27. Input and Output Frequency Response up to 2 kHz

6.2a Performance of Each Edge Detection Implementation: The standard CUDA implementation took 202.21 milliseconds on average to process all four image sizes. The tiled implementation achieved a 1.32x speedup, taking 152.96 milliseconds on average to process all four image sizes. Finally, the implementation loading data to the device as uchar3 data types witnessed the fastest speedup of 1.37x, taking 147.09 milliseconds on average to process all four image sizes. Detailed results for each trial are shown below:

Explorer P100	edge (ms)	edge_tiled (ms)	edge_uchar3 (ms)
Trial 1	269.826806	147.830945	183.737971
Trial 2	189.48407	153.483467	136.306354
Trial 3	191.910378	182.345314	140.166531
Trial 4	176.389408	159.873721	141.24158
Trial 5	183.418933	121.264163	133.975963
Average	202.205919	152.959522	147.0856798

Figure 28. Runtimes for Each Edge Detection Implementation

.2b Performance for Different Image Sizes: The GPU took longer to perform edge detection if the image was larger, but all input samples were processed in under 110 milliseconds. Runtime appears to grow linearly with the number of pixels in the image. Results are shown in the two figures below:

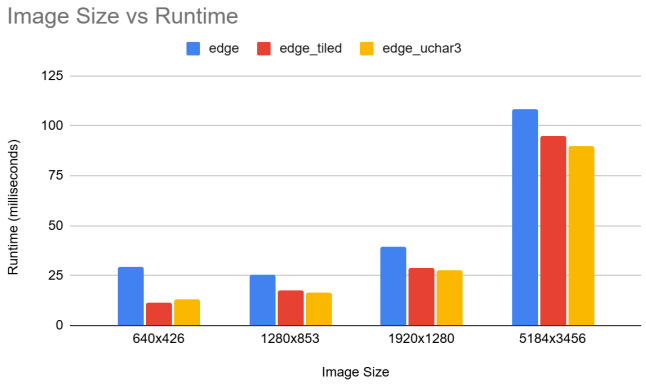

Figure 29. Runtimes for Each Image Size

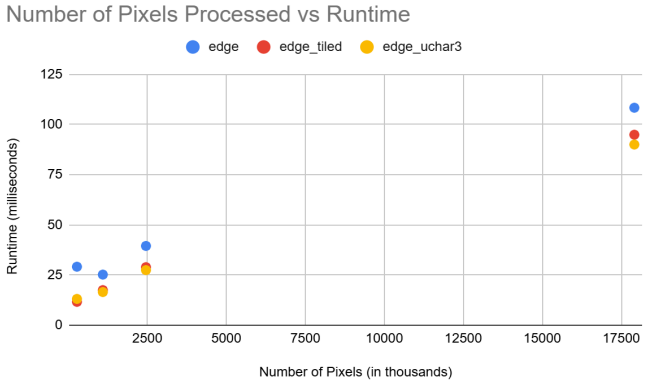

Figure 30. Runtime Compared to Number of Pixels Processed

6.2c Output Image Correctness: Each implementation generated an equivalent set of results. The tree line is clearly highlighted as well as the shooting stars and the rows along the field. It should be noted that the higher resolution output images have more detail, better following the objects, where the lower resolution output images contain more swirled shapes. The 640x426 output image with edges highlighted is shown below:

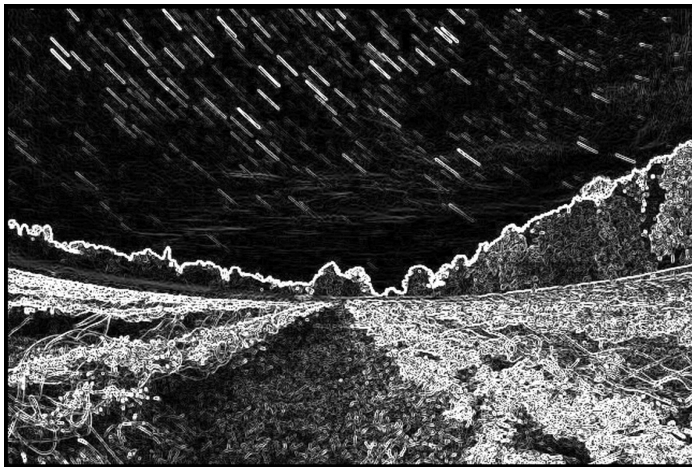

Figure 31. 640x426 Output Image

7. Analysis

7.1a Optimal FIR Filter Implementation when AVX512 Support is Available: As seen below in figure 32, the combination approach clearly outperforms either OpenMP or AVX512 instructions alone. AVX support alone provides about a 6x speedup compared to the theoretically but unattainable 16x speedup with 512-bit registers. AVX support is not available on most CPUs and it consumes more power function. OpenMP alone provides a 35x speedup on up to 48 available cores. OpenMP effectively utilizes parallelism to concurrently process results quickly, and only requires a few extra lines of code. This is a very practical optimization that could be effective on any multi-core processor. In comparison, the single-threaded implementation is cumbersome and significantly slower than any of the parallel versions.

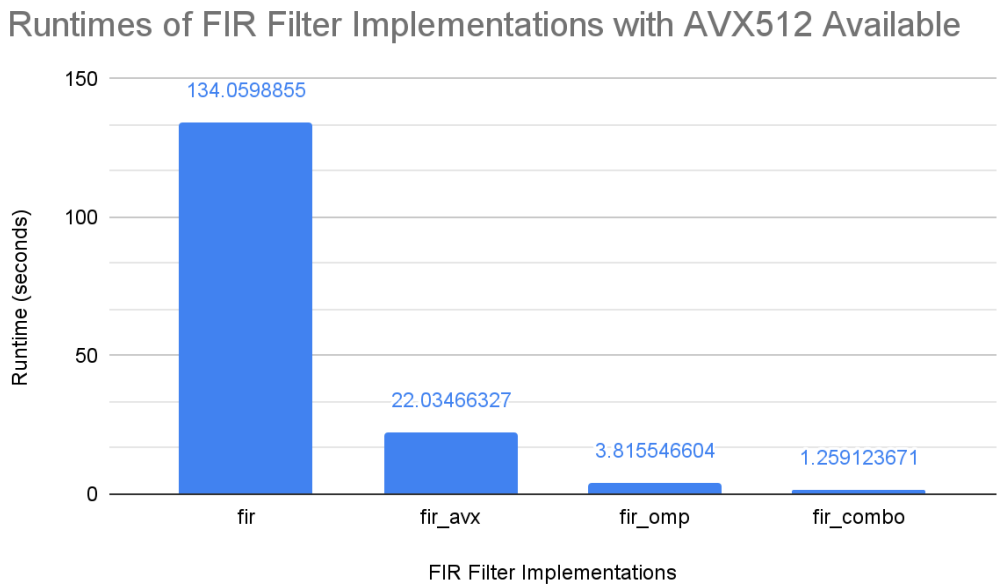

Figure 32. FIR Implementation with AVX Runtime Comparison

7.1b Parallel FIR Filter Implementations without AVX512 Support: As seen below in figure 33, both the Explorer Cluster and COE Vector Systems had significant execution time reductions when OpenMP was applied. The fastest implementation without AVX support was using OpenMP on the Explorer Cluster. This is somewhat surprising since the Xeon E5-2680 on Explorer only has 28 cores available compared to the 40 available on the Xeon E5-2698 on the COE Vector System. This, along with findings in section 6.1d, imply that nodes on the Explorer Cluster may have a lower overhead cost to apply multithreading via OpenMP. However, the E5-2680 also has a slightly higher clock speed of 2.40 GHz compared to the E5-2698's clock speed of 2.20 GHz, which could also contribute to faster computations. Finally, another major contributing factor is that parallelizing the loops parsing stereo data doubled the read/write time on the COE Vector System, taking up approximately 31.94% of the total runtime. When the FIR filter is highly optimized, reading and writing data becomes the bottleneck, so it is critical to ensure this is also done as efficiently as possible for maximum performance.

7.1c Single-Threaded FIR Filter Implementations without AVX Support: Comparing the serial approaches, the COE Vector System significantly outperformed the Explorer Cluster, running about 1.26x faster. One cause is the fact that the Explorer Cluster spent about twice as long to read and write WAV file data. However, since filtration also took longer on the Explorer Cluster, which has a faster clock speed, other differences in architecture or cache latency/size must also contribute to this difference. The figure below depicts these runtimes:

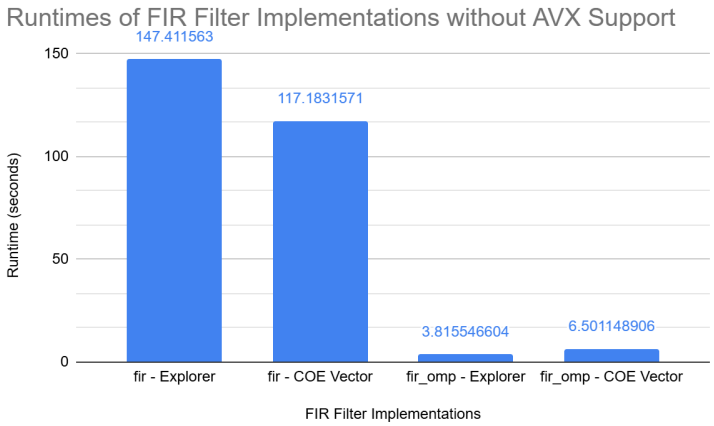

Figure 33. FIR Implementation without AVX Runtime Comparison

7.2a Sobel Filter Edge Detection on a GPU - Tiling and Data Types: Both tiling and using the uchar3 datatype to load RGB pixel values increased performance with about a 1.32-1.37x speedup. The uchar3 optimizations provided slightly faster runtimes than shared memory tiling. Due to this, combining both optimization approaches would likely produce the fastest result. The uchar3 data type allows the GPU device to read an entire RGB pixel into one variable, instead of three separate red, green, and blue values, coallasing the memory access for lower latency. Using shared memory improves performance by improving temporal locality and utilizing the lower latency of shared memory compared to global memory. For optimal performance, the size of the shared memory tile must be chosen carefully to allow the maximum number of threads to work on available data concurrently.

7.2b Sobel Filter Edge Detection on a GPU - Scalability: GPUs were designed for graphics processing, so it makes sense that they would be an advantageous architecture for image processing algorithms such as edge detection. GPU accelerated Sobel filter edge detection workloads can be massively scaled. This is evident because throughput continued to increase as the problem size was increased, meaning more computational work could be completed in the same amount of time compared to a smaller problem. For the largest image size, a throughput of 165 to 200 pixels per microsecond. In other words, GPUs stand out for large scale problems. The chart below demonstrates the weak scaling properties of GPU-based Sobel filter edge detection:

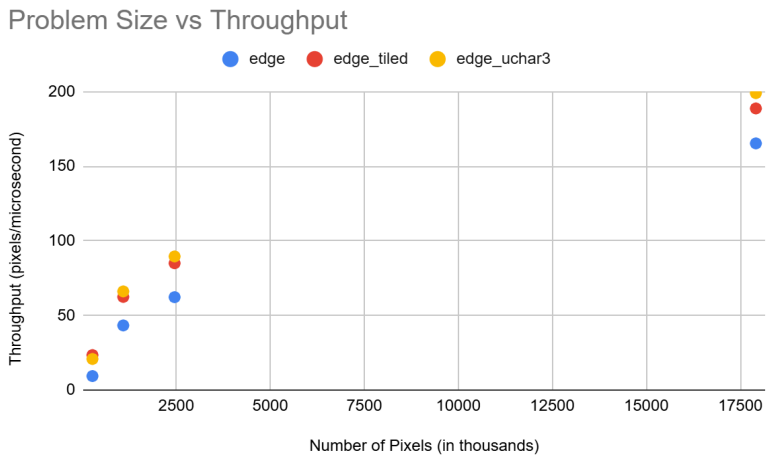

Figure 34. Number of Pixels Compared to Throughput

8. Conclusion

8.1 Takeaways from Parallel FIR Filter Implementations: FIR filtration speed can be massively reduced by taking advantage of parallel computing techniques. An operation that originally took over two minutes using single-threading can be done in just over a second using a combination of AVX512 and OpenMP. When considered alone, OpenMP contributes proportionally more to increasing throughput. Since AVX instructions require careful implementation in source code, require additional modification when considering non-symmetric FIR filters, and only works on CPUs that support AVX512 instructions, the combined approach is best suited for filtering long audio samples or when real time processing is required. In general cases, the simple addition of parallel for loops using OpenMP will drastically speed up code and produce a resulting WAV file in a few seconds. Additionally, total runtime can be bottlenecked by the time to read and write files. It is important to consider the system's architecture to implement the fastest process. Furthermore, data streaming could be added to process audio samples while concurrently loading in the next dataset. Finally, the uniform and independent nature of linear convolutions would also make GPU acceleration an attractive alternative approach to utilize their many cores, but effective decisions about grid size and shared memory usage would need to be considered.

8.2a Scalability and Applications of GPU Accelerated Edge Detection Implementations: Image processing programs, such as edge detection, can be completed very quickly using a GPU implementation, especially if shared memory tiling and/or the uchar3 data type is used to load pixel values. This implementation for Sobel filter edge

detections also achieved a very large maximum throughput of around 200 million pixels per second, proving it is scalable for large images. Using asynchronous copying, this type of throughput could also be achieved when working with videos, concurrently processing multiple frames on as many threads as possible. Edge detection on a video could be very helpful for computer vision applications, highlighting the edges of objects for better identification.

8.2b Quality of the Sobel Filter Method: While the Sobel filter clearly identified the treeline and the streaks in the sky, the field and forest were quite busy due to the many edges of the grass and leaves. The lower part of the output image could be considered overfitted. For a busier image where individual objects/regions are striped or jagged, a either minimum gradient threshold should be added or a more involved edge detection algorithm such as Canny edge detection should be used. The Sobel filter method was chosen for this experiment since it is fairly simple and easy to parallelize.

9. Sources

Akasapu, A., Sailaja, V., & Prasad, G. (2021). Implementation of Sobel filter using CUDA. *IOP Conference Series: Materials Science and Engineering*, *1045*(1), 012016. **https://doi.org/10.1088/1757-899X/1045/1/012016**

Fisher, R., Perkins, S., Walker, A., & Wolfart, E. (2003.). *Sobel Edge Detector*. HIPR2: Hypermedia Image Processing Reference. https://homepages.inf.ed.ac.uk/rbf/HIPR2/sobel.htm

Manolakis, D. G., & Ingle, V. K. (2011). *Applied Digital Signal Processing: Theory and Practice*. Cambridge University Press.